
Practical No.:	05
Practical Title :	To Setup and Run Selenium Tests in Jenkins Using Maven.
Aim:	To automate testing by integrating Selenium tests with Jenkins using Maven for continuous testing.
Objective:	To integrate a GitHub repository with Jenkins for automated builds To configure and execute Selenium test cases using Maven To automate the build process using Jenkins To implement scheduled builds using Jenkins cron triggers

Theory:**What is Jenkins in Selenium?**

Jenkins is an open-source Continuous Integration (CI) tool used to automate software development processes such as building, testing, and deployment. It enables developers to integrate code changes frequently and automatically execute test cases.

Jenkins is platform-independent and works on Windows, Linux, and macOS systems. It integrates with tools like GitHub, Maven, and Selenium to create automated workflows.

Why Jenkins with Selenium?

- Automates execution of test cases
- Enables continuous testing
- Provides build history and logs
- Supports scheduled execution
- Improves software quality

What is Maven?

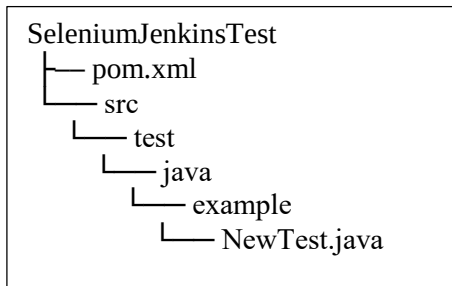
Maven is a build automation and dependency management tool. It uses a configuration file called pom.xml to define project structure, dependencies, and build lifecycle.

Why Maven with Jenkins?

- Manages dependencies automatically
- Standardizes project structure
- Executes test cases using plugins
- Integrates easily with Jenkins

Project Structure:

The project is stored in a GitHub repository with the following structure:



Implementation Steps:

Step 1: Create GitHub Repository

1. Open GitHub
2. Click on New Repository
3. Enter repository name: ***SeleniumJenkinsTest***
4. Add files:
 - pom.xml
 - NewTest.java

Step 2: Add Maven Configuration

Create a file named pom.xml and add the following code:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>example</groupId>
  <artifactId>SeleniumJenkinsTest</artifactId>
  <version>1.0</version>

  <dependencies>

    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.21.0</version>
    </dependency>

    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.10.1</version>
      <scope>test</scope>
    </dependency>

  </dependencies>
```

```
<build>
<plugins>

<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.11.0</version>
<configuration>
<source>21</source>
<target>21</target>
</configuration>
</plugin>

<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>3.2.5</version>
</plugin>

</plugins>
</build>

</project>
```

Step 3: Create Selenium Test Class

Create the file: *'src/test/java/example/NewTest.java'*

Add the following code:

```
package example;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.testng.Assert;
import org.testng.annotations.*;

public class NewTest {

    WebDriver driver;

    @BeforeTest
    public void setup(){

        System.out.println("=====");
        System.out.println("Starting Selenium Test Execution");
        System.out.println("=====");

        String os = System.getProperty("os.name").toLowerCase();
        System.out.println("Detected OS: " + os);
```

```
ChromeOptions options = new ChromeOptions();

if (os.contains("linux")) {
    options.addArguments("--headless=new");
    options.addArguments("--no-sandbox");
    options.addArguments("--disable-dev-shm-usage");
    System.out.println("Running in headless mode");
} else {
    System.out.println("Running in normal mode");
}

driver = new ChromeDriver(options);
}

@Test
public void testTitle() throws InterruptedException {

    System.out.println("Opening website: https://www.rcpit.ac.in");

    driver.get("https://www.rcpit.ac.in");

    String title = driver.getTitle();

    System.out.println("Page title: " + title);

    Assert.assertTrue(title.toLowerCase().contains("rcpit") || title.length() > 0);

    System.out.println("Title verification PASSED");

    System.out.println("Waiting 10 seconds...");
    Thread.sleep(10000);
}

@AfterTest
public void close(){

    System.out.println("Closing browser...");
    driver.quit();

    System.out.println("=====");
    System.out.println("Execution Finished");
    System.out.println("=====");

}

}
```

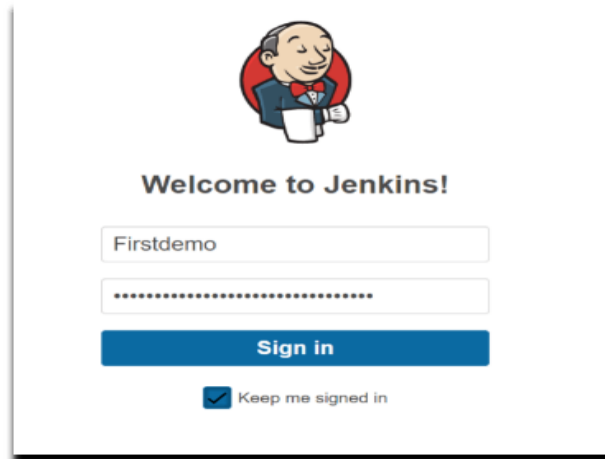
Step 4: Open Jenkins Dashboard

1. Start Jenkins service
-

2. Open browser
3. Go to: '<http://localhost:8080>'

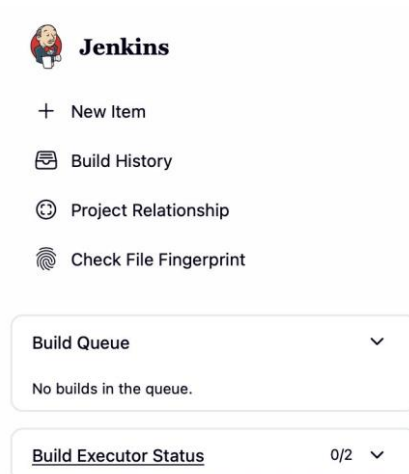
The port number may differ based on the initial Jenkins setup.

4. Login to Jenkins

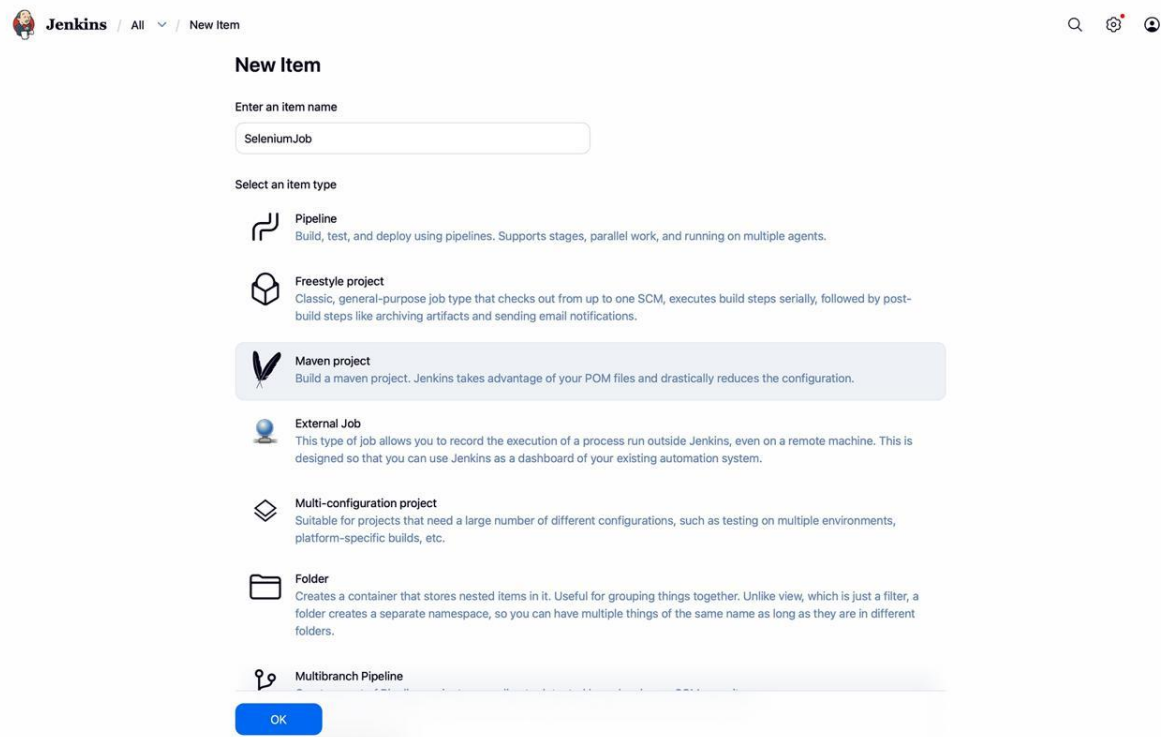


Step 5: Create Jenkins Job

1. Click **New Item**



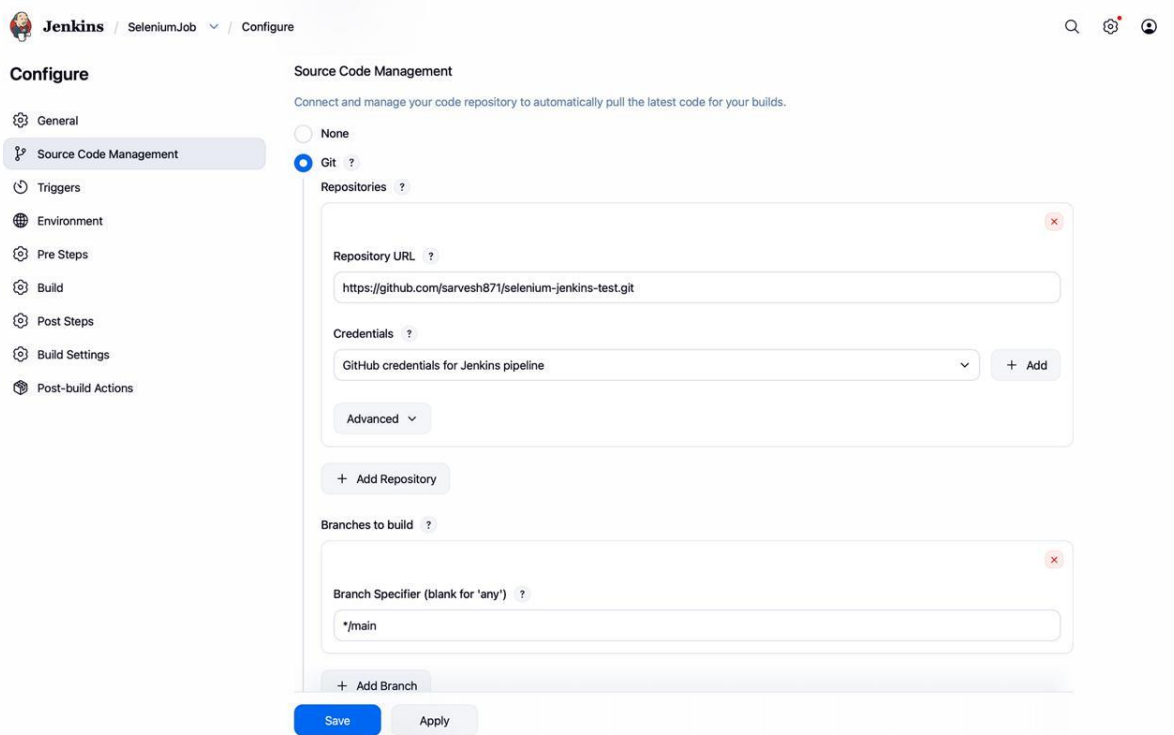
2. Enter name: '**SeleniumJob**'
3. Select **Maven Project**
4. Click **Ok**



The screenshot shows the Jenkins 'New Item' configuration page. At the top, there's a breadcrumb 'Jenkins / All / New Item'. Below it, the 'Enter an item name' field contains 'SeleniumJob'. Under 'Select an item type', several options are listed: Pipeline, Freestyle project, Maven project (highlighted), External Job, Multi-configuration project, Folder, and Multibranch Pipeline. Each option has a brief description. At the bottom, there is an 'OK' button.

Step 6: Connect GitHub Repository

1. Go to **Source Code Management**
2. Select **Git**
3. Enter repository URL
4. Set branch: **'*/main'**



The screenshot shows the Jenkins 'Configure' page for 'SeleniumJob'. The left sidebar has a 'Configure' section with various tabs: General, Source Code Management (selected), Triggers, Environment, Pre Steps, Build, Post Steps, Build Settings, and Post-build Actions. The main area is titled 'Source Code Management' and contains instructions to 'Connect and manage your code repository to automatically pull the latest code for your builds.' There are two radio buttons: 'None' and 'Git' (selected). Below, the 'Repositories' section has a 'Repository URL' field with 'https://github.com/sarvesh871/selenium-jenkins-test.git', a 'Credentials' dropdown set to 'GitHub credentials for Jenkins pipeline', and an 'Advanced' dropdown. There is an '+ Add Repository' button. The 'Branches to build' section has a 'Branch Specifier (blank for \'any\')' field with '*/main' and an '+ Add Branch' button. At the bottom, there are 'Save' and 'Apply' buttons.

Step 7: Configure Build

In the Build section:

Root POM: pom.xml

Goals: clean test

Build

Root POM ?

pom.xml

Goals and options ?

clean test

Advanced ▾

Step 8: Configure Build Trigger

1. Scroll to Build Triggers
2. Enable: *'Build whenever a SNAPSHOT dependency is built'*
3. Enable: *'GitHub hook trigger for GITScm polling'*
4. Enable: *'Build periodically'*
5. Enter: *'*/2 * * * *'*

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☒ Build whenever a SNAPSHOT dependency is built ?

☐ Schedule build when some upstream has no successful builds ?

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☒ Build periodically ?

Schedule ?

*/2 * * * *

⚠ Spread load evenly by using 'H/2 * * * *' rather than '*/2 * * * *'

Would last have run at Sunday, 22 March, 2026, 9:20:00pm India Standard Time; would next run at Sunday, 22 March, 2026, 9:22:00pm India Standard Time.

☒ GitHub hook trigger for GITScm polling ?

☐ Poll SCM ?

Step 9: Run the Build

1. Click Build Now
2. Jenkins will:
 - Fetch code from GitHub
 - Build project using Maven
 - Execute Selenium test

Jenkins / SeleniumJob

Status

</> Changes

Workspace

Build Now

Configure

Delete Maven project

Modules

GitHub Hook Log

Rename

SeleniumJob

Latest Test Result (no failures)

Permalinks

- Last build (#2), 1 min 0 sec ago
- Last stable build (#2), 1 min 0 sec ago
- Last successful build (#2), 1 min 0 sec ago
- Last completed build (#2), 1 min 0 sec ago

Test Result Trend

1

0.5

#2

REST API Jenkins 2.541.2

Step 10: View Console Output

1. Click on build number
2. Click Console Output

Jenkins / SeleniumJob / #3

SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See <http://www.slf4j.org/codes.html#StaticLoggerBinder> for further details.

Starting Selenium Test Execution

Detected OS: mac os x
Running in normal mode (GUI available)
Mar 22, 2026 10:14:12 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 146
Mar 22, 2026 10:14:12 PM org.openqa.selenium.chromium.ChromiumDriver lambda\$new\$5
WARNING: Unable to find version of CDP to use for 146.0.7680.153. You may need to include a dependency on a specific version of the CDP using something similar to 'org.seleniumhq.selenium:selenium-devtools-v86:4.21.0' where the version ("v86") matches the version of the chromium-based browser you're using and the version number of the artifact is the same as Selenium's.
Opening website: <https://www.rcpit.ac.in>
Page title: R. C. Patel Institute of Technology, Shirpur | An Autonomous Institute
Title verification PASSED
Waiting 10 seconds...
Closing browser...

Execution Finished

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 19.62 s -- in example.NewTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[JENKINS] Recording test results
[INFO] BUILD SUCCESS
[INFO] Total time: 24.575 s
[INFO] Finished at: 2026-03-22T22:14:29+05:30
[INFO]
Waiting for Jenkins to finish collecting data
[JENKINS] Archiving /Users/admin/.jenkins/workspace/SeleniumJob/pom.xml to example/SeleniumJenkinsTest/1.0/SeleniumJenkinsTest-1.0.pom
channel stopped
Finished: SUCCESS

Benefits of using Jenkins:

1. Automates testing process
2. Detects errors early
3. Maintains build history
4. Supports continuous integration
5. Integrates with GitHub and Maven

Conclusion:
